



Felvételi, 2024. szeptember
– Alapképzés, informatika vizsga –
JAVÍTÓKULCS

Hivatalból 1 pont jár. Munkaidő 3 óra.

Megjegyzések:

- A pszeudokód algoritmusban
 - Az „=” értékadást, az „==” pedig egyenlőség vizsgálatot jelent
 - A / operátorral osztási hányadost, a % operátorral pedig osztási maradékot kapunk meg
 - A „minden $i = 1$, n végezd” jelentése: minden $i = 1, 2, 3, \dots, n$ értékre végezd el...
- A programokat megjegyzésekkel kell ellátni!
- Törekedjünk hatékony megoldásokra! (maximális pontszám feltétele)

1. (1 pont) Legyen az alábbi pszeudokód programrészlet:

```
res = 0
amíg n ≠ 0 végezd
| res = res + (n % 2) + (n % 3)
| n = n / 10
■
kiír res
```

Mit ír ki a fenti programrészlet, ha $n = 20240910$?

13

2. (1 pont) Legyen az alábbi pszeudokód programrészlet:

```
res = 0
amíg n ≠ 0 végezd
| ha (n % 10) == num akkor
| | res = res * 10 + (n % 10)
| ■
| különben
| | res = res * 10 + 9 - (n % 10)
| ■
| n = n / 10
| ■
kiír res
```

Mit ír ki a fenti programrészlet, ha $n = 20240910$ és $num = 2$?

98095292

3. (1 pont) Legyen az alábbi pszeudokód programrészlet:

```
i = 1
amíg i ≠ 4 végezd
| ha (i % 2) == 0 akkor
| | num = (num % 10) + i
| ■
| különben
| | num = (num % 10) + (i * 2)
| ■
| i = i + 1
| ■
kiír num
```

Mit ír ki a fenti programrészlet, ha $num = 2024$?



4. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről három, pontosan kétszámjegyű természetes számot, majd kiírja az első szám számjegyeit növekvő, a második szám számjegyeit pedig csökkenő sorrendben, a harmadik szám számjegyeinek pedig az összegét, külön-külön sorba, egy-egy szóközzel elválasztva egymástól.

Példák:

Bemenet:
13 47 53

Kimenet:
1 3
7 4
8

Bemenet:
21 33 10

Kimenet:
1 2
3 3
1

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;

    int elsoA = a / 10, masodikA = a % 10;
    int elsoB = b / 10, masodikB = b % 10;
    int elsoC = c / 10, masodikC = c % 10;

    if (elsoA > masodikA) {
        cout << masodikA << " " << elsoA << endl;
    } else {
        cout << elsoA << " " << masodikA << endl;
    }

    if (elsoB > masodikB) {
        cout << elsoB << " " << masodikB << endl;
    } else {
        cout << masodikB << " " << elsoB << endl;
    }

    cout << elsoC + masodikC << endl;
    return 0;
}
```

5. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről három egész számot (ezek egy háromszög oldalainak hosszát jelentik), majd eldönti és kiírja, hogy a megadott számokkal lehet-e háromszöget szerkeszteni. Egy háromszög akkor szerkeszthető, ha bármely két oldal hosszának összege nagyobb, mint a harmadik oldal hossza.

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;

    if (a + b > c && a + c > b && b + c > a) {
        cout << "A megadott oldalhosszakkal szerkeszthető háromszög." << endl;
    } else {
        cout << "A megadott oldalhosszakkal nem szerkeszthető háromszög." << endl;
    }

    return 0;
}
```



6. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről egy n ($1 \leq n \leq 100$) természetes számot, majd egy n elemű, egész számokat tartalmazó számsorozatot. A program végezze el a következőket:

- Számolja meg és írja ki, hogy hány olyan szám található a számsorozatban, amely pozitív és osztható 3-mal! (0,5 pont)
- Írja ki a számsorozat elemeit a képernyőre a beolvasás sorrendjében, minden második számot 0-val helyettesítve! (0,5 pont)

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int tomb[n];

    for (int i = 0; i < n; ++i) {
        cin >> tomb[i];
    }

    // Számolás
    int szamlalo = 0;

    for (int i = 0; i < n; i++) {
        if (tomb[i] > 0 && tomb[i] % 3 == 0) {
            szamlalo++;
        }
    }

    cout << "Pozitív és 3-mal osztható számok száma: " << szamlalo << endl;

    // Számsorozat kiírása
    cout << "A módosított számsorozat:" << endl;

    for (int i = 0; i < n; i++) {
        if ((i + 1) % 2 == 0) {
            cout << 0 << " ";
        } else {
            cout << tomb[i] << " ";
        }
    }

    cout << endl;
    return 0;
}
```



7. (1 pont) Írj Pascal vagy C/C++ **eljárást** (void-függvényt), amely paraméterként megkapja az n értéket ($1 \leq n \leq 100$), valamint egy egydimenziós tömböt, amely egy n -elemű egész számsorozatot tárol. Az eljárás eldönti, hogy egy adott n -elemű tömb az $\{1, 2, \dots, n\}$ halmaz egy permutációját tartalmazza-e. A program írjon ki egy IGEN/NEM üzenetet.

Példák:

Bemenet:

5
4 1 3 2 5

Kimenet:

IGEN

Bemenet:

4
4 1 3 3

Kimenet:

NEM

```
void permutacioEllenorzes(int n, int tomb[])
{
    // Egy n-elemű bool tömb, amely a számok előfordulását jelzi
    bool volt[101] = {false}; // n+1, mert az indexelés 1-től indul

    for (int i = 0; i < n; ++i)
    {
        // Ellenőrizzük, hogy az adott szám az 1 és n közötti tartományban van-e
        if (tomb[i] < 1 || tomb[i] > n)
        {
            cout << "NEM" << endl;
            return;
        }

        // Ha a szám már előfordult, akkor nem lehet permutáció
        if (volt[tomb[i]])
        {
            cout << "NEM" << endl;
            return;
        }

        // Jelöljük meg, hogy a tomb[i] érték már előfordult
        volt[tomb[i]] = true;
    }

    // Ha minden szám egyszer fordult elő, akkor permutáció -> IGEN
    cout << "IGEN" << endl;
}
```

8. (1 pont) Írj Pascal vagy C/C++ **programot**, amely egy bemeneti állományból (*input.txt*) beolvassa az n értéket (ahol $2 \leq n \leq 50$) és egy $n \times n$ természetes számokat tartalmazó mátrixot. A programnak el kell döntenie, hogy a mátrix keretein található elemek összegeiből álló sorozat növekvő és páros-e, ahogy a külső kerettől haladunk befelé. Ha a keretek összegei növekvők és párosak, akkor a program az "IGEN" szót írja ki, különben "NEM"-et.

Példa:

Bemenet:

```
6
2  4  6  8  5  1
4  6  8 10 15  2
6  8 10 12  5  1
8 10 12 14  5  9
1  9 21 23  2  4
2  1  1  1  1  1
```

Kimenet:

NEM

Magyarázat: A harmadik keret összege habár páros, de kisebb, mint az előzőé.



```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int main() {
    ifstream input("input.txt");
    int n;

    input >> n;
    int matrix[n][n];

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            input >> matrix[i][j];
        }
    }
    // Előző keret összege, kezdetben -1 (ami azt jelzi, hogy nincs előző keret)
    int elozoOsszeg = -1;

    for (int keret = 0; keret < (n + 1) / 2; ++keret) {
        int aktualisOsszeg = 0;

        // Felső sor
        for (int i = keret; i < n - keret; ++i) {
            aktualisOsszeg += matrix[keret][i];
        }

        // Jobb oszlop
        for (int i = keret + 1; i < n - keret; ++i) {
            aktualisOsszeg += matrix[i][n - keret - 1];
        }

        // Alsó sor
        for (int i = n - keret - 2; i >= keret; --i) {
            aktualisOsszeg += matrix[n - keret - 1][i];
        }

        // Bal oszlop
        for (int i = n - keret - 2; i > keret; --i) {
            aktualisOsszeg += matrix[i][keret];
        }

        // A keret összege nem páros vagy nem nő
        if (aktualisOsszeg % 2 != 0 || (elozoOsszeg != -1 && aktualisOsszeg <=
elozoOsszeg))
        {
            cout << "NEM" << endl;
            return 0;
        }

        // Az előző keret összegét frissítjük az aktuálisra
        elozoOsszeg = aktualisOsszeg;
    }

    // Ha minden feltétel teljesült, kiírjuk az IGEN-t
    cout << "IGEN" << endl;
    return 0;
}
```



9. (1 pont) Írj Pascal vagy C/C++ **függvényt**, amely egy $n \times n$ méretű kétdimenziós tömbben egy robot mozgását szimulálja (ahol $2 \leq n \leq 99$). A robot “középről” indul (index: $n/2$, $n/2$), és mindig az aktuális pozíció értéke alapján mozog. Ha az aktuális pozícióban lévő szám *pozitív*, akkor a robot *jobbra és egyet fel* mozog, ha pedig *negatív*, akkor *balra és egyet le*. A robot addig mozog, amíg ki nem lép a tömbből, vagy csapdába nem esik egy olyan pozícióban, ahol már járt. A robot minden lépés során annyi pozíciót ugrik jobbra vagy balra, amennyi az aktuális pozícióban található *szám abszolút értéke*.

Ha a robot ki tud lépni a tömbből, akkor a program térítse vissza a lépések számát, beleértve azt is amellyikkel ténylegesen kilépett (kilépésnek tekintjük, ha a robot egyik indexe a $[0, n-1]$ tartományon kívül esik). Ha a robot csapdába esik, akkor a program térítse vissza a -1 értéket. A függvény paraméterként megkapja a tömb méretét (n), és magát a tömböt.

Példa:

Bemenet:

```
6
7 1 2 -1 -4 4
3 5 2 4 1 -2
1 -2 -3 -4 -5 3
9 5 -3 -1 3 -3
2 4 3 -2 8 -9
1 -6 -8 7 4 -3
```

Kimenet:

-1

Magyarázat:

Az útvonal (kezdőérték pozíciója: $(6/2, 6/2)$, azaz a $(3, 3)$ -as mező):

$(3, 3)$ értéke: $-1 \rightarrow$ balra lépünk 1-et és 1-et le

$(4, 2)$ értéke: $3 \rightarrow$ jobbra lépünk 3-at és 1-et fel

$(3, 5)$ értéke: $-3 \rightarrow$ balra lépünk 3-at és 1-et le

$(4, 2)$ értéke: $3 \rightarrow$ itt már jártunk - A program kimenete: -1

Amennyiben a $(3, 5)$ pozíción 3-as állna, akkor a program kimenete a lépések száma lenne:
3

```
#include <cmath>
using namespace std;

int robotMozgas(int n, int matrix[100][100]) {
    // Kezdő pozíció a mátrix közepén (n / 2, n / 2)
    int x = n / 2; // Sor index
    int y = n / 2; // Oszlop index

    // Látogatott mezők
    bool latogatott[n][n];

    // Kezdetben egyik mezőt sem látogattuk még meg
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            latogatott[i][j] = false;
        }
    }

    int lepesek = 0;

    // A robot addig mozog, amíg nem lép ki a mátrixból vagy nem esik csapdába
    while (true) {
        if (latogatott[x][y]) {
            // Csapdába lépett
            return -1;
        }
        // Mozgás
        int dx = 0, dy = 0;
        if (matrix[x][y] > 0) {
            dx = 1; // jobbra
            dy = 1; // fel
        } else {
            dx = -1; // balra
            dy = -1; // le
        }
        int nx = x + dx, ny = y + dy;
        if (nx < 0 || nx > n-1 || ny < 0 || ny > n-1) {
            // Kilépett
            return lepesek + 1;
        }
        latogatott[x][y] = true;
        x = nx; y = ny;
        lepesek++;
    }
}
```




```
}

latogatott[x][y] = true;
lepesek++;

int ertekek = matrix[x][y];

// A robot mozgása az érték alapján: y irányba az érték szerint (jobbra-balra),
// x irányba pedig felfelé vagy lefelé, az érték előjelétől függően
y += ertekek;
x += (ertekek > 0 ? -1 : 1);

// Ha a robot kilépett a mátrix határain kívülre, visszatérünk a megtett lépések
számával
if (x < 0 || x >= n || y < 0 || y >= n) {
    return lepesek;
}
}
```